Date: .....

Expt. No.: ..... Page No.: .....

SQL Tutorial for Beginners (MySQL)

- **Database**

> A database is a way to store data in a format which makes it easily accessible.

- **DBMS**

> DBMS stands for Database Management System.

> DBMS generally help us to perform CRUD operation on database which are create, read, update and delete.

- **RDBMS**

> RDBMS stands for Relational database Management System.

> follows a fixed and predefined schema.

> Data is organized into structured tables with rows and columns.

> Ensure data integrity through constraints and relationship.

> Utilize SQL (structured Query Language) for defining and manipulating data.

> Well-suited for applications requiring complex queries and transactional consistency.

> Examples are MySQL, PostgreSQL

Teacher's Signature _____

• Non-Relational Database Management System (NOSQL)

- It provides flexible data storage and modelling.
- It also does not enforce a fixed or consistent schema.
- SQL is not mandatory for data manipulation and querying.
- Ideal for use cases where read operations are more frequent than write operations.
- Examples are MongoDB, Cassandra, Redis etc.

• Database

- A database is a container that stores related data in an organized way. In MySQL, a database holds one or more tables.

Think of it like:

- Folder analogy:
  - A database is like a folder
  - Each table is a file inside that folder
  - The rows in the table are like the content inside each file.

- Excel analogy:
  - A database is like an Excel workbook.
  - Each table is a separate sheet inside that workbook
  - Each row in a table is like a row in Excel

Date: .....

Expt. No.: .....

Page No.: .....

Step 1:

Create a Database

```
CREATE DATABASE starkmysql;
```

after clicking on creating the database, either:

- Right-click it in MySQL workbench and select "Set as default schema" or
- Use this SQL command:

```
USE startmysql;
```

Step 2:

Create a Table

```
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    gender ENUM ('Male', 'Female', 'Other'),
    date_of_birth DATE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
SELECT * FROM users;
```

Step 3:

Drop the Database

You can delete the entire database (and all its tables) using:

```
DROP DATABASE starkmysql;
```

Teacher's Signature _____

• DataTypes Explained

- INT : Integer type, used for whole numbers.
- VARCHAR(100): Variable-length string, up to 100 characters
- ENUM : A string object with a value chosen from a list of permitted values. eg.,

  gender ENUM('Male', 'Female', 'Other')
- DATE : Stores date values eg., date-of-birth DATE
- TIMESTAMP : Stores date and time, automatically set to the current timestamp when a row is created.

• Constraints Explained

- AUTO_INCREMENT : Automatically generates a unique number for each row.
- PRIMARY_KEY : Uniquely identifies each row in the table.
- NOT NULL : Ensures a column cannot have a NULL value.
- UNIQUE : Ensures all values in a column are different.
- DEFAULT : Sets a default value for a column if no value is provided eg., created_at TIMESTAMP,

  DEFAULT CURRENT_TIMESTAMP

Date: .....

Expt. No.: .....

Page No.: .....

- Selecting Data From Table

> Select all columns

SELECT * FROM users;

This fetches every column and every row from the users table.

> Specific Select columns

SELECT name, email FROM users;

This only fetches the name and email columns and all rows from it.

> Renaming a Table

to rename an existing table:

RENAME TABLE users TO customers;

to rename it back:

RENAME TABLE customers TO users;

- Altering a Table.

> You can use ALTER TABLE to modify an existing table.

> Add a Column

ALTER TABLE Users ADD COLUMN is_active BOOLEAN DEFAULT TRUE;

> Drop a Column

ALTER TABLE Users DROP COLUMN is_active;

> Modify a Column Type

ALTER TABLE users MODIFY COLUMN name VARCHAR(150);

Teacher's Signature _____

- Move a Column to the First Position

to move a column (eg., email) to the first position:

ALTER TABLE users MODIFY COLUMN email VARCHAR(150) FIRST;

to move column after another column (e.g., move gender after name):

ALTER TABLE users MODIFY COLUMN gender ENUM('Male', 'Female', 'Others') AFTER name;

★ Inserting Data Into MySQL Tables

To add data into a table, we use INSERT INTO statement.

- Insert Without Specifying Column Names (Full Row Insert)

↳ This method requires you to provide values for all columns in order, except columns with default values or AUTO_INCREMENT.

```
INSERT INTO users VALUES
(1, [redacted], 'singhmanip@gmail.com', 'Manip', 'Male', '2005-05-2
, DEFAULT );
```

↳ Not recommended if your table structure might change (e.g., new columns added later).

- Insert by Specifying Column Names (Best Practice)

↳ This method is safer and more readable. you only insert into specific columns.

```
INSERT INTO users (name, email, gender, date_of_birth) VALUE
('Mohit', 'Mohit18@gmail.com', 'Male', '2006-08-31');
```

Date: .....

Expt. No.: ..... Page No.: .....

- Insert Multiple Rows at Once by Specifying Column Names

> INSERT INTO users (name, email, gender, date_of_birth)
VALUES
('Alice', 'alice@gmail.com', 'Male', '2005-01-20'),
('Sadhya', 'saahya01@gmail.com', 'Female', '2004-03-21'),
('Bob', 'bob28@gmail.com', 'Mal', '2003-05-15');

> This is more efficient than Inserting rows one by one.
> The remaining columns like id (which is AUTO_INCREMENT) and created_at (which has a default) are automatically handled by MySQL.

★ Querying Data in MySQL Using Select

> The SELECT statement is used to query data from a table.

- Basic Syntax
Select column1, column2 FROM table_name;
> to select all columns:
SELECT * FROM users;

- Filtering Rows with WHERE
> Equal to
SELECT * FROM Users WHERE gender = 'Male';
> Not Equal to
SELECT * FROM users WHERE gender != 'Female';
-- or
SELECT * FROM users WHERE gender <> 'Female';

Teacher's Signature _____

- Greater than / less than

```
SELECT * FROM users WHERE date_of_birth < '2005-01-01';
SELECT * FROM users WHERE id > 10;
```

- Greater than or Equal / less than or Equal

```
SELECT * FROM users WHERE id >= 5
SELECT * FROM users WHERE id <= 20
```

• Working with NULL

- IS NULL

```
SELECT * FROM users WHERE date_of_birth IS NULL;
```

- IS NOT NULL

```
SELECT * FROM users WHERE date_of_birth IS NOT NULL;
```

• BETWEEN

```
SELECT * FROM users WHERE date_of_birth BETWEEN
'2000-01-01' AND '2005-01-01';
```

• IN

```
SELECT * FROM users WHERE gender IN ('MALE', 'Other');
```

• LIKE (Pattern Matching)

```
SELECT * FROM users WHERE name LIKE 'A%'; -- Starts with A
SELECT * FROM users WHERE name LIKE '%a'; -- Ends with a
SELECT * FROM users WHERE name LIKE '%li%' -- Contains li
```

• AND / OR

```
SELECT * FROM users WHERE gender = 'Female' AND
date_of_birth > '2000-01-01';
```

Date: .....

Expt. No.: ..... Page No.: .....

> SELECT * FROM users WHERE gender = 'male' OR gender = 'other';

- ORDER BY

> SELECT * FROM users ORDER BY date_of_birth ASC;
> SELECT * FROM users ORDER BY date_of_birth DESC;

- LIMIT

> SELECT * FROM users LIMIT 5; -- Top 5 rows
> SELECT * FROM users LIMIT 10 OFFSET 5; -- Skip first 5 rows, then get next 10.
> SELECT * FROM users LIMIT 5,10; -- Get 10 rows starting from 6th rows.
> SELECT * FROM users ORDER BY created_at DESC LIMIT 10;

- Quick Quiz

> What does the following Queries do ?
SELECT * FROM users WHERE salary > 60000 ORDER BY created_at DESC LIMIT 5;
SELECT * FROM users ORDER BY salary DESC;
SELECT * FROM users WHERE salary BETWEEN 50000 AND 70000;

Teacher's Signature _____

★ Update - Modifying Existing Data

↳ The UPDATE statement is used to change values in one or more rows.

• Basic Syntax

```
UPDATE table_name
SET column1 = value1, column2 = value2
WHERE condition;
```

Example : update One column

```
UPDATE users
SET name = 'Alicia'
WHERE id = 1;
```

Example : Update Multiple Columns

```
UPDATE users
SET name = 'Robert', email = 'robert@example.com'
WHERE id = 2;
```

• Without WHERE Clause (warning)

```
UPDATE users
SET gender = 'Other';
```

↳ This update every rows in the table. Be very careful when omitting WHERE clause.

Expt. No.: ..... Page No.: .....

• Quick Quiz: Practice your UPDATE skills

1) update the salary new column in your existing table first

2) gives them specific salary.

3) update the salary of user with id = 5 to Rs. 70,000

4) change the name of the user with email manipsingh @gmail.com to Manip Singh.

5) Increase salary by Rs. 10000 for all users whose salary is less than Rs. 50000.

6) Set the gender of user Bob to other.

7)

★ Delete - Removing Data from table

> The DELETE statement removes rows from a table

• Basic Syntax

```
DELETE FROM table_name
WHERE condition;
```

Example: Delete One Row

```
DELETE FROM users
WHERE id = 3;
```

Example: Multiple Rows

```
DELETE FROM users
WHERE gender = 'Others';
```

Teacher's Signature _____

- Delete All Rows (but keep table structure)
  - DELETE FROM users;

- Delete The Entire Table (use with caution)
  - DELETE TABLE users;

★ MySQL Constraints

- Constraints in MySQL are rules applied to table columns to ensure the accuracy, validity and integrity of the data

1. UNIQUE Constraints
- Ensures that all values in a column are different.

Example (during table creation):

```
CREATE TABLE users (
        id INT PRIMARY KEY,
        email VARCHAR(100) UNIQUE
);
```

- Add UNIQUE using ALTER TABLE:
  - ALTER TABLE users ADD CONSTRAINT unique_email UNIQUE (email);

2. NOT NULL Constraint
- Ensures that a column cannot contain NULL values.

Example:

```
CREATE TABLE users (
        id INT PRIMARY KEY,
        name VARCHAR(100) NOT NULL
);
```

Date: .....

Expt. No.: .....

Page No.: .....

- Change an existing column to NOT NULL:

ALTER TABLE users
MODIFY COLUMN name VARCHAR(100) NOT NULL;

- Make a column nullable again:

ALTER TABLE users
MODIFY COLUMN name VARCHAR(100) NULL;

3. CHECK Constraint

> Ensure that values in a column satisfy a specific condition.

Example: Allow only dates of birth after Jan 1, 2000.

ALTER TABLE users
ADD CONSTRAINT chk_dob CHECK (date_of_birth > '2000-01-01');

4. DEFAULT constraint

> Sets a default value for a column if none is provided during insert.

Example:

CREATE TABLE users (
    id INT PRIMARY KEY,
    is_active BOOLEAN DEFAULT TRUE
);

- Add default using Alter a Table

ALTER TABLE users MODIFY COLUMN is_active SET DEFAULT TRUE;

Teacher's Signature _____

5. PRIMARY KEY constraint

> Uniquely identifies each row. Must be NOT NULL and unique.

Example:

```
CREATE TABLE users (
        id INT PRIMARY KEY,
        name VARCHAR(100)
);
```

6. AUTO-INCREMENT constraint

> Used with PRIMARY KEY to automatically assign the next number.

Example:

```
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100)
);
```

> Each new row gets the next available integer value in id.

Expt. No.: ..... Page No.: .....

★ SQL Functions (MySQL)

> SQL Functions help you to analyse, transform or summarize data in your tables.

> We will use the users table which includes:
- id, name, email, gender, date-of-birth, salary, created at

1. Aggregate Functions

- COUNT() : Count total number of users.

```
SELECT COUNT(*) FROM users;
SELECT COUNT(*) FROM users WHERE gender = 'Female';
```

- MIN() and MAX() : Get the Minimum and Maximum Salary:

```
SELECT MIN(salary) as MIN-salary, MAX(salary as
Max-salary FROM users;
```

- SUM() : Calculate total salary payout.

```
SELECT SUM(salary) as total-payroll FROM users;
```

- AVG() : And Average Salary

```
SELECT AVG(salary) AS avg-salary FROM users;
```

- GROUP BY : Averageing Salary by gender.

```
SELECT gender, AVG(salary) as avg-salary FROM users
GROUP By gender;
```

Teacher's Signature _____

2. String Functions

- LENGTH() : Length of user names.

SELECT name, LENGTH(name) AS name_length FROM users;

- LOWER() and UPPER() : Convert names to lowercase and uppercase.

SELECT name, LOWER(name) AS lowercase_name FROM users;
SELECT name, UPPER(name) AS uppercase_name FROM users;

- CONCAT() : Combine name and email

SELECT CONCAT(name, '<', email, '>') AS user_contact FROM users;

3. Date Functions

- NOW() : Current date and time.

SELECT NOW();

- YEAR(), MONTH(), DAY() : Extract parts of date_of_birth

SELECT name, YEAR(date_of_birth) AS birth_year FROM users;
SELECT name, MONTH(date_of_birth) AS birth_month FROM users;
SELECT name, DAY(date_of_birth) AS birth_day FROM users;

- DATEDIFF() : Find the no of difference b/w today and birth

SELECT name, DATEDIFF(CURDATE(), date_of_birth) AS days_lived FROM users.

- TIMESTAMPDIFF() : Calculate age in years.

SELECT name, TIMESTAMPDIFF(YEAR, date_of_birth, CURDATE())
AS age FROM users;

Date: .....

Expt No.: ..... Page No.: .....

4. Mathematical Functions

- ROUND(), FLOOR(), CEIL() : use for float values

```
SELECT salary,
        ROUND(salary) AS rounded,
        FLOOR(salary) AS floored,
        CEIL(salary) AS ceiled
FROM users;
```

- MOD() : Find even or odd user IDs.

```
SELECT id, MOD(id,2) AS remainder FROM users;
```

5. CONDITIONAL Functions

- IF()

```
SELECT name, gender,
        IF(gender = 'Female', 'yes', 'No') AS is-female
FROM users;
```

★ MySQL Transactions and AutoCommit ★

1. Disabling AutoCommit

> When Autocommit is off, you can explicitly control when to commit or rollback changes.

```
SET autocommit = 0;
```

2. COMMIT - Save changes to the Database

> Once you have made changes and you are confident that everything is correct, you can use COMMIT command

```
COMMIT;
```

Teacher's Signature _____

3. ROLLBACK - Revert changes to the last safe Point
> if you make an error or decide you don't want to save your changes, you can rollback the transaction to the previous state

ROLLBACK;

Example:

```
SET autocommit = 0;
UPDATE users SET Salary = 80000 WHERE id = 5;
COMMIT;
-- OR
ROLLBACK;
DELETE ~~users~~ FROM users WHERE id = 5;
COMMIT;
```

4. Enable Autocommit Again:
> SET Autocommit = 1;

★ PRIMARY KEY in MySQL

> A PRIMARY KEY is a constraint in SQL that uniquely identifies each row in a table. It is one of the most important concepts in database design.

Date: .....

Page No.: .....

Expt. No.: .....

- What is Primary key ?
  - > Must be unique
  - > Cannot be NULL
  - > Is used to identify rows in table
  - > can be a single column or a combination of column
  - > Each table can have only one primary key.

Example :

```
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100)
);
```

- How Primary key is Different from UNIQUE

feature	PRIMARY KEY	UNIQUE
> Must be unique	Yes	Yes
> Allows NULL values	NO	Yes (one or more NULLs allowed)
> How many allowed	Only one per table	can have multiple
> Required by table	Recommended	optional
> Dropping	Cannot be dropped easily	Can be dropped anytime

Teacher's Signature _____

★ FOREIGN KEYS

> A foreign key is a column that creates a link between two tables. It ensures that the value in one table must match a value in another table.
> This is used to maintain data integrity between related data.

- Why Use Foreign Keys ?

Imagine this scenario :
You have a users table. Now you want to store each users addresses. Instead of putting address columns inside the users table, you create a separate addresses table and link it to users using foreign key.

- Adding ON DELETE Action

By default if you will delete a user that has related addresses, MySQL will throw an error. You can control this behaviour with ON DELETE :

Example :

```
CREATE TABLE addresses (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT,
    street VARCHAR(255),
    city VARCHAR(100),
    state VARCHAR(100),
    pincode VARCHAR(10),
    CONSTRAINT fk_user FOREIGN KEY (user_id) REFERENCES
        users(id) ON DELETE CASCADE
);
```

Expt. No.: ..... Page No.: .....

- OR alter if later:

ALTER TABLE addresses
ADD CONSTRAINT fk_user FOREIGN KEY (user_id) REFERENCES
users (id) ON DELETE CASCADE;

- Other ON DELETE options:
  - CASCADE: Deletes all related rows in child table
  - SET NULL: Sets the foreign key to NULL in the child table
  - RESTRICT: Prevents deletion of parent if child exists (default).

- Dropping a foreign key
  - To drop a foreign key, you need to know its constraint name. Mysql auto-generate it if you don't specify one or you can name it yourself.

ALTER TABLE addresses
DROP FOREIGN KEY fk_user;

Teacher's Signature _____

★ SQL JOINs in MySQL

→ In SQL, JOINs are used to combine rows from two or more tables based on related columns - usually a foreign key in one table referencing a primary key into another.

users table

id	name
1	Aarav
2	Sneha
3	Ray

addresses table

id	user_id	city
1	1	Mumbai
2	2	Kolkata
3	4	Delhi

1. INNER JOIN

→ Returns only matching rows from both tables.

```
SELECT users.name, addresses.city
FROM users
INNER JOIN addresses ON users.id = addresses.user_id;
```

2. LEFT JOIN

→ Returns all rows from the left table (users), and matching rows from the right table (addresses).
if no match is found, NULLs are returned

```
SELECT users.name, addresses.city
FROM users
LEFT JOIN addresses on users.id = addresses.user_id;
```

Date: .....

Expt. No.: ..... Page No.: .....

3. RIGHT JOIN

> Returns all rows from the right table (addresses) and matching rows from the left table (users). If no match is found, NULLs are returned.

```
SELECT users.name, addresses.city
FROM users
RIGHT JOIN addresses ON users.id = addresses.user_id;
```

- Output

INNER JOIN		LEFT JOIN		RIGHT JOIN	
name	city	name	city	name	city
Aaav	Mumbai	Aaav	Mumbai	Aaav	Mumbai
Sneha	Kolkata	Sneha	Kolkata	Sneha	Kolkata
		Raj	NULL	NULL	Delhi

★ SQL UNION and UNION ALL

> The UNION operator in SQL is used to combine the result sets of two or more SELECT statements. It removes duplicates by default.

> if you want to include all rows including duplicates USE UNION ALL.

Teacher's Signature _____

Step 1: Create the admin_users Table

```
CREATE TABLE admin_users (
        id INT PRIMARY KEY,
        name VARCHAR(100),
        email VARCHAR(100),
        gender ENUM('Male', 'Female', 'Other'),
        date_of_birth DATE,
        salary INT
);
```

Step 2: INSERT Sample Data into admin_users

```
INSERT INTO admin_users (id, name, email, gender, date_of_birth
                        , salary) VALUES
    (101, 'Anil Kumar', 'anil@example.com', 'Male', '1985-04-12'
                        , 60000),
    (102, 'Pooja Sharma', 'pooja@example.com', 'Female', '1992-02-29'
                        , 58000);
```

Step 3: USE UNION to Combine Data

```
SELECT name FROM users
UNION
SELECT name FROM admin_users
```

Step 4: UNION ALL

```
SELECT name FROM users
UNION ALL
SELECT name FROM admin_users;
```

Date: .....

Expt. No.: .....

Page No.: .....

- Using More than One Column

```
SELECT name, salary, gender FROM users
UNION
SELECT name, salary, gender FROM admin_users;
```

- Using ORDER BY with UNION

```
SELECT name FROM users
UNION
SELECT name FROM admin_users
ORDER BY name;
```

★ Self Join in MySQL

- A self join is a regular join, but the table is joined with itself.

- This is useful when rows in the same table are related to each other. For example, when users refer other users, and we store the ID of the person who referred them in the same users table.

Step 1: Add a referred_by_id column

```
ALTER TABLE users
ADD COLUMN referred_by_id INT;
```

- This column will be NULL for users who were not referred.
- Will contain the id of another users who referred them.

Teacher's Signature _____

Step 2: Insert Referral Data (optional)
Assume id = 1 is the first user and referred others

```
UPDATE users SET referred_by_id = 1 WHERE id IN (2, 5, 7, 9);
UPDATE users SET referred_by_id = 2 WHERE id IN (3, 10, 15);
```

Step 3: Use a self join to get referrer names
I we want to get each user's name along with the name of the person who referred them.

```
SELECT
    a.id,
    a.name as user_name,
    b.name as referred_by
FROM users a
LEFT JOIN users b ON a.referred_by_id = b.id;
```

★ MySQL Views

> A view in MySQL is a virtual table based on the result of a SELECT query. It does not store data itself. It always reflects the current data in the base tables.

> Views are useful when:
- you want to simplify complex queries
- you want to reuse logic
- you want to hide certain columns from users
- you want a "live snapshot" of filtered data.

Date: .....

Expt. No.: .....

Page No.: .....

Step 1: Creating a View

```
CREATE VIEW high_salary_users AS
SELECT id, name, salary
FROM users
WHERE salary > 70000;
```

Step 2: Querying the View

```
SELECT * FROM high_salary_users;
```

Step 3: Update a user's salary

```
UPDATE users
SET salary = 72000
WHERE id = 1;
```

Step 4: Querying the View Again

```
SELECT * FROM high_salary_users;
```

Step 5: Dropping a View

```
DROP VIEW high_salary_users;
```

★ MySQL Indexes

→ Indexes in MySQL are used to speed up data retrieval. They work like the index of a book - helping the database engine find rows faster, especially for large tables, filters and joins.

Teacher's Signature _____

Step 1: Viewing Indexes on a Table

SHOW INDEXES FROM users;

> This shows all the indexes currently defined on the users table. Including the automatically created Primary Key index.

Step 2: Creating a Single-Column Index

CREATE INDEX idex_email ON users (email);

> Creates an index named idex_email

> Improves performance of queries like:

SELECT * FROM users WHERE email = 'example@example.com';

Step 3: Creating a Multi-Column Index

CREATE INDEX idex_gender_salary ON users (gender, salary);

> Usage Example:

SELECT * FROM users

WHERE gender = 'Female' AND salary > 70000;

Step 4: Dropping an Index

DROP INDEX idex_email ON users;

- INDEX ORDER matters
  - for a multi-column index on (gender, Salary):
    - > this works efficiently

      WHERE gender = 'Female' AND salary > 70000;

    - > But this may not use index efficiently:

      WHERE Salary > 70000;

Date: .....

Expt. No.: ..... Page No.: .....

• Important Notes

> Indexes consumes extra disk space

> Indexes slow down INSERT, UPDATE and DELETE operations slightly (because the index must be updated)

> Use Indexes only when needed (i.e., for column used WHERE, JOIN + ORDER BY).

★ Subqueries in MySQL

> A Subquery is a query nested inside another query. Subqueries are useful for breaking down complex problems into smaller parts.

> They can be used in:
- SELECT statements
- WHERE clauses
- FROM clauses

> Example Scenario: Salary Comparison
Suppose we want to find all users who earn more than the average salary of all users.

• Scalar Subquery Example

> This subquery returns a single value - the average salary - and we compare each users salary against it.

```
SELECT id, name, salary
FROM users
WHERE salary > ( SELECT AVG(salary) FROM users );
```

Teacher's Signature _____

- **Subquery with IN**

> Now let's say we want to find users who have been referred by someone who earns more than Rs. 75,000.

```
SELECT id, name, referred_by_id
FROM users
WHERE referred_by_id IN (
        SELECT id FROM users WHERE salary > 75000
);
```

- **Other Places Subqueries Are Used**

> You can also use subqueries:
  - Inside SELECT columns (called scalar subqueries).
  - In the FORM clause to create derived tables.

Example in SELECT:

```
SELECT name, salary,
        (SELECT AVG(salary) FROM users) AS average_salary
FROM users;
```

> This shows each user's salary along with the overall average.

Date: .....

Expt. No.: ..... Page No.: .....

★ GROUP BY and HAVING in MySQL

> The GROUP BY clause is used to group rows that have the same values in specified columns. It is typically used with aggregate functions like COUNT, SUM, AVG, MIN or MAX.

> Example Table: users

Assume this is your users table:

id	name	gender	salary	referred_by_id
1	Aakar	Male	80000	NULL
2	Sneha	Female	75000	1
3	Raj	Male	72000	1
4	Fatima	Female	85000	2
5	Priya	Female	70000	NULL

> GROUP BY Example: Average Salary by Gender

```
SELECT gender, AVG(salary) AS average_salary
FROM users
GROUP BY gender;
```

> GROUP BY with COUNT

find how many users were referred by each user.

```
SELECT referred_by_id, COUNT(*) AS total_referred
FROM users
WHERE referred_by_id IS NOT NULL
GROUP BY referred_by_id;
```

Teacher's Signature _____

• HAVING Clause : Filtering Groups

> Let's say we only want to show genders where the average salary is greater than ₹. 75000.

```
SELECT gender, AVG(salary) AS avg_salary
FROM users
GROUP BY gender
HAVING AVG(salary) > 75000;
```

1 Another Example : Groups with More than 1 Referral

```
SELECT referred_by_id, COUNT(*) AS total_referred
 FROM users
WHERE referred_by_id IS NOT NULL
GROUP BY referred_by_id
HAVING COUNT(*) > 1;
```

★ Stored Procedures in MySQL

> A Stored Procedure is a saved SQL block that can be executed later. It's useful when you want to encapsulate logic that can be reused multiple times - like queries, updates, or conditional operations

• Why change the Delimiter ?

> By default MySQL uses ; to end SQL statements;

> But when defining stored procedures we use ; inside the procedure as well. This can confuse MySQL. To avoid this, we temporarily change the delimeter (e.g to $$ or //)

Date: .....

Expt. No.: .....

Page No.: .....

★ Syntax for Creating a Stored Procedure

```
DELIMETER $$
CREATE PROCEDURE Procedure_name (
BEGIN
    IN p_name VARCHAR (100),
    IN p_email VARCHAR (150),
    IN p_gender ENUM ('Male', 'Female', 'Other'),
    IN p_dob DATE,
    IN p_salary INT.
)
BEGIN
    INSERT INTO users (name, email, gender, date_
        of_birth, salary)
        VALUES (p_name, p_email, p_gender, p_dob
            , p_salary);
        SELECT * FROM users;
END $$
DELIMETER;

CALL procedure_name ('Bear', 'bear26@example.com',
    'Male', '2002-12-07', 66000);
SHOW PROCEDURE STATUS WHERE Db = 'storeusql';
```

Teacher's Signature _____

★ Trigger in MySQL

> A trigger is a special kind of stored Program that is automatically executed (triggered) when a specific event occurs in a table - such as INSERT, UPDATE, or DELETE

> Triggers are commonly used for:

- Logging changes
- Enforcing additional business rules
- Automatically updating related data

• Basic Trigger Structure

Step 1: Create a log Table

```
CREATE TABLE user_log (
        id INT AUTO_INCREMENT PRIMARY KEY,
        user_id INT,
        name VARCHAR(100),
        created_on TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Step 2: Create the Trigger

```
DELIMITTER $$
CREATE TRIGGER after_user_insert
AFTER INSERT on users
FOR EACH ROW
BEGIN
        INSERT INTO user_log ( user_id, name)
         VALUES (NEW.id, NEW.name);
END $$
DELIMITTER;
```

Date: .....

Expt. No.: .....

Page No.: .....

Step 3: Test The Trigger

CALL Procedure_name ('Rihka Jain', 'rihka@mample.com', 'Female', '1996-03-12', 75000);

OR

INSERT INTO users (name, email, gender, date_of_birth, salary)
VALUES ('Rohan', 'rohanxrohan@example.com', 'Male', '2007-04-04', 56000);

> Now check the user_log Table:
SELECT * FROM user_log;

Step 4: Dropping a Trigger.

> If you need to Remove trigger.
DROP TRIGGER IF EXISTS after_user_insert;

Teacher's Signature _____